

For a triplet to form a geometric sequence, it has to adhere to two main rules:

1. It consists of three values that follow a geometric sequence with a common ratio r .
2. The three values forming the triplet must appear in the same order within the array as they do in the geometric sequence. This means for a geometric triplet $(\text{nums}[i], \text{nums}[j], \text{nums}[k])$, the indexes must follow the order $i < j < k$.

How can we represent a geometric sequence so that it follows rule 1? Let's say the first number is x . The second number is the first number multiplied by r (i.e., $x \cdot r$), and the third is the second number multiplied by r (i.e., $x \cdot r \cdot r = x \cdot r^2$). So, a triplet in a geometric sequence can be represented as $(x, x \cdot r, x \cdot r^2)$.

A brute force approach is to iterate over every possible triplet in the array and check if any of these triplets follow a geometric progression. However, it would take three nested for-loops to search through all the triplets, resulting in a time complexity of $O(n^3)$, where n denotes the length of the input array. Can we do better?

An important observation here is that if we know one value of a triplet, we can calculate what the other two values should be.

This is because all three values are related by the common ratio r . So, for any number x in the array, we just need to find the values $x \cdot r$ and $x \cdot r^2$ to form a geometric triplet $(x, x \cdot r, x \cdot r^2)$. However, we could run into issues when using this triplet representation. While it's clear the values $x \cdot r$ and $x \cdot r^2$ must be positioned to the right of x , we have to be careful since the order of these values matters: we don't want to accidentally identify a triplet such as $(x, x \cdot r^2, x \cdot r)$, which is invalid:

$$\begin{array}{c} x \\ \downarrow \\ [\cdot \quad \cdot \quad x \quad \cdot \quad \cdot \quad x \cdot r^2 \quad \cdot \quad x \cdot r] \\ \text{triplet values } (x, x \cdot r^2, x \cdot r) \text{ appear in the wrong order} \end{array}$$

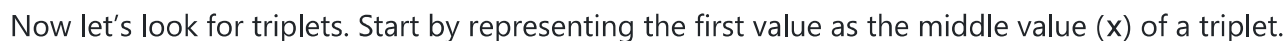
We can work around this issue by using the $(x/r, x, x \cdot r)$ triplet representation, which allows us to always maintain order by looking for x/r to the left of x and $x \cdot r$ to the right:

$$\left[\underbrace{\cdot \quad \cdot \quad \cdot \quad \cdot}_{\text{find } x/r} \right] \quad x \quad \left[\underbrace{\cdot \quad \cdot \quad \cdot}_{\text{find } x \cdot r} \right]$$

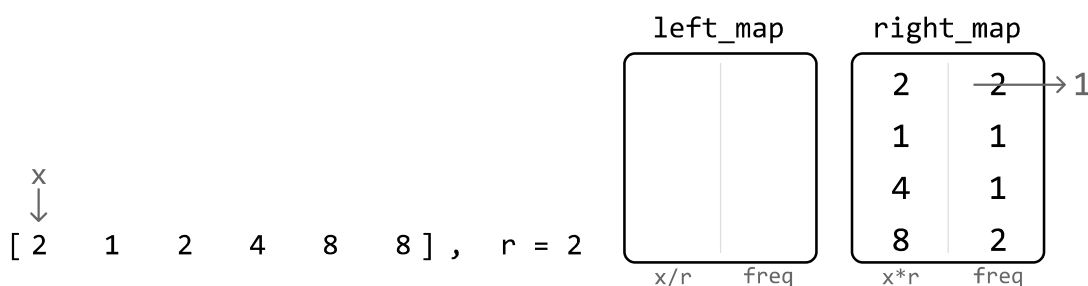
One way we can find the x/r and $x \cdot r$ values is by linearly searching through the left and right subarrays. This linear search would need to be done for each number in the array, resulting in an $O(n^2)$ time complexity. While this is an improvement from the brute force solution, it would be great if we had a way to find those values faster.


$$\begin{bmatrix} 2 & 1 & 2 & 4 & 8 & 8 \end{bmatrix}, \quad r = 2$$

Since we're traversing the array from left to right, we should **initially fill the right hash map with all values in the array**. This is because, before the start of the iteration, every element is a potential candidate for x/r . Meanwhile, the left hash map is initially empty because there are no preceding elements to consider as potential x/r values:



First, let's **update right_map**. We should remove the current value (2) from **right_map** since this 2 is not to the right of itself. There are two 2's in **right_map**, so let's reduce its frequency to 1:



hash maps. Since `left_map` doesn't contain x/r at this point, its frequency is 0:

x
 $\downarrow$
`[ 2    1    2    4    8    8 ]`
 $r = 2$

```
x % r == 0 → count += left_map[x/r] * right_map[x*r]
               += left_map[2/2] * right_map[2*2]
               += 0 * 1
               += 0
```

Before moving on to the next value, let's **add the current number to the `left_map`** because it now becomes a potential x/r value for future triplets in the array:

left_map		right_map	
2	1	2	1
		1	1
		4	1
		8	2
x/r	freq	$x*r$	freq

Repeating this process for the rest of the array allows us to find all geometric triplets with a ratio of r . To clarify, the hash maps in the upcoming diagrams represent their state at the current position of x in the array. This means that `left_map` includes values to the left of the current x , and the `right_map` includes values to the right of it.

x
 $\downarrow$
`[ 2    1    2    4    8    8 ]` , $r = 2$

left_map		right_map	
2	1	2	1
		1	0
		4	1
		8	2
x/r	freq	$x*r$	freq

```
x % r != 0 → can't form a triplet → continue
```

left_map		right_map	
2	1	2	0
1	1	1	0
		4	1
		8	2
x/r	freq	x*r	freq

```
x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[2/2] * right_map[2*2]
                += 1 * 1
                += 1
```

$x \downarrow$
[2 1 2 4 8 8], r = 2

left_map		right_map	
2	2	2	0
1	1	1	0
		4	0
		8	2
x/r	freq	x*r	freq

```
x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[4/2] * right_map[4*2]
                += 2 * 2
                += 4
```

$x \downarrow$
[2 1 2 4 8 8], r = 2

left_map		right_map	
2	2	2	0
1	1	1	0
4	1	4	0
		8	1
x/r	freq	x*r	freq

```
x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[8/2] * right_map[8*2]
                += 1 * 0
                += 0
```

$x \downarrow$
[2 1 2 4 8 8], r = 2

left_map		right_map	
2	2	2	0
1	1	1	0
4	1	4	0
8	1	8	0
x/r	freq	x*r	freq

```
x%r == 0 → count += left_map[x/r] * right_map[x*r]
                += left_map[8/2] * right_map[8*2]
                += 1 * 0
                += 0
```

Implementation

```
def geometric_sequence_triplets(nums: List[int], r: int) → int:
    # Use 'defaultdict' to ensure the default value of 0 is returned when
    # accessing a key that doesn't exist in the hash map. This effectively sets
    # the default frequency of all elements to 0.
    left_map = defaultdict(int)
    right_map = defaultdict(int)
    count = 0
    # Populate 'right_map' with the frequency of each element in the array.
    for x in nums:
        right_map[x] += 1
    # Search for geometric triplets that have x as the center.
    for x in nums:
        # Decrement the frequency of x in 'right_map' since x is now being
        # processed and is no longer to the right.
        right_map[x] -= 1
        if x % r == 0:
            count += left_map[x // r] * right_map[x * r]
        # Increment the frequency of x in 'left_map' since it'll be a part of the
        # left side of the array once we iterate to the next value of 'x'.
        left_map[x] += 1
    return count
```

Complexity Analysis

Time complexity: The time complexity of `geometric_sequence_triplets` is $O(n)$ because we iterate through the `nums` array and perform constant-time hash map operations at each iteration.

Space complexity: The space complexity is $O(n)$ because the hash maps can grow up to n in size.

☐ Mark as Complete

[Teach on ByteByteGo](#)

[Be an Affiliate](#)

[Become a Contributor](#)



[Report a Bug](#)

[Our Team](#)

[Newsletter](#)